

dirichlet_dist_bayesian_update_pt1-oc

May 15, 2026

1 Bayesian Inference for the Multinomial Distribution

In this exercise, you will write your own Python code to perform two instances of Bayesian update.

1.1 Bayesian Update

Recall that Bayes' rule states that a posterior probability is proportional to the product of the likelihood and the prior, such that: $posterior \propto likelihood \times prior$.

1.1.1 The Prior

Let's say we have some data from a previous experiment that we base as our prior. If we have some prior distribution on μ taking the form of a Dirichlet distribution, the prior, $p(\vec{\mu}|\vec{\alpha}) = Dir(\vec{\mu}|\vec{\alpha})$, is equal to $\frac{\Gamma(\alpha_0)}{\Gamma(\alpha_1)\dots\Gamma(\alpha_K)} \prod_{k=1}^K \mu_k^{(\alpha_k-1)}$. Here, the hyperparameters $\vec{\alpha}$ are counts of the number of 1's and 0's generated in a prior Multinomial trial of N_{prior} samples. So for example, if we rolled 30d4 and found that we got 12 ones, 4 twos, 8 threes, and 6 fours, then we would establish that $K = 4$, $\alpha_1 = 12$, $\alpha_2 = 4$, $\alpha_3 = 8$, and $\alpha_4 = 6$. Here, $N_{prior} = \alpha_0 = \sum_{k=1}^K \alpha_k$.

1.1.2 The Likelihood

Now let's consider the likelihood term, which captures new information. Let's say we perform a new experiment and get new data (new information). We roll another $N_{likelihood} = 10$ d4. In this trial, there were an additional $m_1 = 4$ ones, $m_2 = 1$ twos, $m_3 = 3$ threes, and $m_4 = 2$ fours.

$N_{likelihood} = \sum_{k=1}^K m_k = 4 + 1 + 3 + 2 = 10$. The parameters m_k captures the number of 1's of each class or "bin" in this new data set.

The likelihood, $p(\vec{\mu}|\vec{m})$, is given by $Mult(m_1, \dots, m_K | (\vec{\mu}, N)) = \binom{N}{m_1 m_2 \dots m_K} \prod_{k=1}^K \mu_k^{m_k}$.

1.1.3 The Posterior

The posterior, $p(\vec{\mu}|\vec{\mathcal{D}}, \vec{\alpha})$ takes on the form of the prior, because the Dirichlet distribution is a conjugate prior! We can use this for sequential estimation. Here the term $\vec{\mathcal{D}}$ captures the new data. $\vec{m}$ captures the counts of each class in $\vec{\mathcal{D}}$.

In the video, Peter showed that the posterior is given by:

$$p(\vec{\mu} | (\vec{\mathcal{D}}, \vec{\alpha})) = \frac{\Gamma(\alpha_0 + N)}{\Gamma(\alpha_1 + m_1) \dots \Gamma(\alpha_K + m_K)} \prod_{k=1}^K \mu^{\alpha_k + m_k - 1},$$

which does indeed take on the form of a Dirichlet distribution.

1.1.4 Bayesian Update

Using the expression for the posterior, we can write a function that can do sequential estimation. In this lab exercise, you will visualize this for the case of $K = 3$, but you should think about how this analysis could be extended to, say, $K = 6$, or $K = 10,000$.

1.2 Visualizing the Bayesian Update

In this part of the lab, assume $K = 3$. Implement Bayesian update for the Dirichlet distribution with the following iterations and plot the response.

1.2.1 Iteration 0:

We start by forming our prior:

$$\vec{m} = [3, 3, 15].$$

1.2.2 Iteration 1:

We get new data to incorporate into our model:

$$\vec{m} = [4, 30, 2].$$

1.2.3 Iteration 2:

We get one final set of new data to incorporate into our model:

$$\vec{m} = [55, 3, 5].$$

1.2.4 Instructions

Rather than trying to plot the distribution on the same set of axes, we will use three sub plots to visualize the data. Please use the starter code provided for plotting. You should also refer back to the previous assignment, where you visualized the Dirichlet distribution for your reference. You are welcome to use any of the code you would like there.

1.3 Write and Run Your Own Code:

Like last time, you should store your iterations of the Bayesian update in an $N \times N \times i$ matrix, where i is the current iteration. This should be passed to a function that computes the Bayesian update and returns the values of the posterior, the updated α parameters, and the iteration, for indexing.

A plotting function is provided for you, that shows you how to use subplots. Call this function at the end, once you have completed all of your iterations of the Bayesian Update. The library calls are also given to you.

1.3.1 Import Key Libraries

```
[1]: import numpy as np

%matplotlib widget
import matplotlib.pyplot as plt
from matplotlib.ticker import LinearLocator, FormatStrFormatter
from matplotlib import cm
from mpl_toolkits.mplot3d import axes3d

from scipy.special import gamma, factorial
```

1.3.2 Create your Plotting Grid

Write and run your own code (though feel free to borrow from code given to you in previous programming exercises—this is ok!).

If you are using starter code you didn't write though, cite it and make sure you understand how it works!

```
[2]: ## Please write and run your code here
# Please add cells as needed
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import gamma

# 1. Define the Dirichlet Probability Density Function (PDF)
def dirichlet_pdf(mu, alpha):
    """
    Computes the Dirichlet PDF for a given set of probabilities mu and
    ↪ parameters alpha.
    mu: array-like of shape (3,) where sum(mu) == 1
    alpha: array-like of shape (3,)
    """
    # Convert mu to a numpy array to allow element-wise operators
    mu = np.asarray(mu)
    alpha = np.asarray(alpha)

    if np.any(mu <= 0) or not np.isclose(np.sum(mu), 1.0, atol=1e-3):
        return 0.0

    # Calculate normalization constant
    B = np.prod(gamma(alpha)) / gamma(np.sum(alpha))

    # Calculate unnormalized density
    density = np.prod(mu ** (alpha - 1))

    return density / B
```

```

# 2. Define the Sequential Bayesian Update Function
def bayesian_update(prior_alpha, data_m):
    """
    Performs a single step of the Bayesian update.
    Returns the updated posterior alpha parameters.
    """
    return np.array(prior_alpha) + np.array(data_m)

# 3. Initialize the Parameters for Iterations
# Iteration 0: Initial Prior Parameters
alpha_0 = np.array([3, 3, 15])

# Iteration 1 Data
m_1 = np.array([4, 30, 2])
alpha_1 = bayesian_update(alpha_0, m_1)

# Iteration 2 Data
m_2 = np.array([55, 3, 5])
alpha_2 = bayesian_update(alpha_1, m_2)

# Print tracking history
print(f"Iteration 0 (Prior Alpha):      {alpha_0}")
print(f"Iteration 1 (Posterior Alpha): {alpha_1}")
print(f"Iteration 2 (Posterior Alpha): {alpha_2}")

# 4. Create Grid and Evaluate Densities for 3D Visualizations
grid_size = 50
x = np.linspace(0.01, 0.99, grid_size)
y = np.linspace(0.01, 0.99, grid_size)
X, Y = np.meshgrid(x, y)

# Initialize the 3D matrix to store grid data for 3 iterations (Grid x Grid x 3)
Z_history = np.zeros((grid_size, grid_size, 3))
alphas_history = [alpha_0, alpha_1, alpha_2]

for idx, alpha in enumerate(alphas_history):
    for i in range(grid_size):
        for j in range(grid_size):
            mu1 = X[i, j]
            mu2 = Y[i, j]
            mu3 = 1.0 - mu1 - mu2

            if mu3 > 0:
                Z_history[i, j, idx] = dirichlet_pdf([mu1, mu2, mu3], alpha)
            else:

```

```

Z_history[i, j, idx] = np.nan

# 5. Plotting Code Using Subplots
fig = plt.figure(figsize=(18, 5))
titles = [
    f"Iteration 0: Prior Alpha {alpha_0}",
    f"Iteration 1: Posterior Alpha {alpha_1}",
    f"Iteration 2: Posterior Alpha {alpha_2}"
]

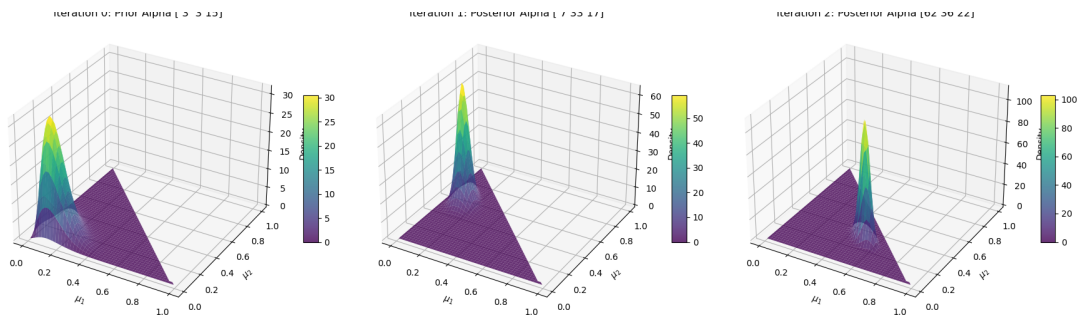
for idx in range(3):
    ax = fig.add_subplot(1, 3, idx + 1, projection='3d')
    # Mask out NaN values for clean rendering
    Z_masked = np.ma.masked_invalid(Z_history[:, :, idx])

    surf = ax.plot_surface(X, Y, Z_masked, cmap='viridis', edgecolor='none',
        alpha=0.8)
    ax.set_title(titles[idx], fontsize=12)
    ax.set_xlabel(r'$\mu_1$')
    ax.set_ylabel(r'$\mu_2$')
    ax.set_zlabel('Density')
    fig.colorbar(surf, ax=ax, shrink=0.5, aspect=10)

plt.tight_layout()
plt.show()

```

Iteration 0 (Prior Alpha): [3 3 15]
 Iteration 1 (Posterior Alpha): [7 33 17]
 Iteration 2 (Posterior Alpha): [62 36 22]



1.3.3 Define a Function that Implements the Bayesian Update

Write and run your own code:

```
[3]: ## Please write and run your code here
# Please add cells as needed
def compute_bayesian_update(grid_X, grid_Y, current_alpha, data_m,
    iteration_idx, Z_matrix):
    """
    Computes the Bayesian update and evaluates the posterior distribution
    density.

    Parameters:
    - grid_X, grid_Y: 2D meshgrid arrays for  $\mu_1$  and  $\mu_2$ .
    - current_alpha: The alpha array BEFORE this update step.
    - data_m: The newly observed count vector (m) for this step.
    - iteration_idx: Integer index of the current iteration (0, 1, 2, ...).
    - Z_matrix: The 3D matrix of shape (N, N, Total_Iterations) storing the
    densities.

    Returns:
    - Z_matrix: Updated 3D density matrix containing the new iteration's slice.
    - updated_alpha: The newly calculated posterior alpha parameters.
    - iteration_idx: The current iteration index for tracking.
    """
    # 1. Update alpha parameters: posterior = prior + observed counts
    updated_alpha = np.asarray(current_alpha) + np.asarray(data_m)

    # 2. Extract grid dimensions (N x N)
    N = grid_X.shape[0]

    # 3. Evaluate the Dirichlet PDF across the parameter space grid
    for i in range(N):
        for j in range(N):
            mu1 = grid_X[i, j]
            mu2 = grid_Y[i, j]
            mu3 = 1.0 - mu1 - mu2

            # The probabilities must be strictly positive and valid
            if mu3 > 0:
                Z_matrix[i, j, iteration_idx] = dirichlet_pdf([mu1, mu2, mu3],
                    updated_alpha)
            else:
                Z_matrix[i, j, iteration_idx] = np.nan

    return Z_matrix, updated_alpha, iteration_idx
```

1.3.4 Define a Plotting Function

You can use this starter code, you shouldn't need to make any modifications, though you should read through it and understand how it works.

```

[4]: ## =====
## This function plots the Dirichlet Distribution:
## =====
## mu_x and mu_y are N_matrix_dim x N_matrix_dim numpy arrays
## That implement a triangular plotting region, and implicitly
## define mu_z

## dirichlet_dist is a N_matrix_dim x N_matrix_dim x 3 matrix that
## holds an N_matrix_dim x N_matrix_dim frame for each iteration,
## starting from iteration 0, which sets the prior. This will let us
## create multiple subplots with this function.

def plot_K3_Dirichlet_dist(mu_x,mu_y,dirichlet_dist):
    ## =====
    ## Find the min and max of the distribution for the color bar
    ## =====
    dist_min = np.nanmin(dirichlet_dist)
    dist_max = np.nanmax(dirichlet_dist)

    ## =====
    ## Create a new 3D figure:
    ## =====
    fig = plt.figure(figsize=(10,4))

    ## =====
    ## Plot Iteration 0
    ## =====
    ax1 = fig.add_subplot(1, 3, 1, projection='3d')
    surf1 = ax1.plot_surface(mu_x, mu_y, dirichlet_dist[:, :, 0], cmap=cm.
↪cividis, linewidth=0, alpha = 0.5, vmin = dist_min, vmax = dist_max)
    ax1.set_title(r'Iteration 0', pad=20)
    ax1.set_xlabel(r'$\mu_{1}$')
    ax1.set_ylabel(r'$\mu_{2}$')
    ax1.set_zlabel(r'$Dir(\vec{\mu}|\vec{\alpha})$', rotation='horizontal')
    ax1.set_ylim(1,0)
    ax1.set_xlim(0,1)

    ## =====
    ## Plot Iteration 1
    ## =====
    ax2 = fig.add_subplot(1, 3, 2, projection='3d')
    surf2 = ax2.plot_surface(mu_x, mu_y, dirichlet_dist[:, :, 1], cmap=cm.
↪cividis, linewidth=0, alpha = 0.5, vmin = dist_min, vmax = dist_max)
    ax2.set_title(r'Iteration 1', pad=20)
    ax2.set_xlabel(r'$\mu_{1}$')
    ax2.set_ylabel(r'$\mu_{2}$')

```

```

ax2.set_zlabel(r'$Dir(\vec{\mu}|\vec{\alpha})$',rotation='horizontal')
ax2.set_ylim(1,0)
ax2.set_xlim(0,1)

## ++++++
## Plot Iteration 2
## ++++++
ax3 = fig.add_subplot(1, 3, 3, projection='3d')
surf3 = ax3.plot_surface(mu_x, mu_y, dirichlet_dist[:, :, 2], cmap=cm.
↪cividis, linewidth=0, alpha = 0.5, vmin = dist_min, vmax = dist_max)
ax3.set_title(r'Iteration 2', pad=20)
ax3.set_xlabel(r'$\mu_1$')
ax3.set_ylabel(r'$\mu_2$')
ax3.set_zlabel(r'$Dir(\vec{\mu}|\vec{\alpha})$',rotation='horizontal')
ax3.set_ylim(1,0)
ax3.set_xlim(0,1)

return

```

1.3.5 Implement your Bayesian Update

Write and Run Your Own Code:

Please use `N_matrix_dim = 128` for your plots, so you can check your numbers accurately in Part 2 of this assignment.

```

[5]: N_matrix_dim = 128
x_vals = np.linspace(0.001, 0.999, N_matrix_dim)
y_vals = np.linspace(0.001, 0.999, N_matrix_dim)
X, Y = np.meshgrid(x_vals, y_vals)

# Initialize the 128x128x3 matrix
Z_history_matrix = np.zeros((N_matrix_dim, N_matrix_dim, 3))

# --- Iteration 0: Prior Setup ---
alpha_0 = np.array([3, 3, 15])
m_0 = np.array([0, 0, 0])
Z_history_matrix, alpha_0, _ = compute_bayesian_update(X, Y, alpha_0, m_0, 0,
↪Z_history_matrix)

# --- Iteration 1: Update 1 ---
m_1 = np.array([4, 30, 2])
Z_history_matrix, alpha_1, _ = compute_bayesian_update(X, Y, alpha_0, m_1, 1,
↪Z_history_matrix)

# --- Iteration 2: Update 2 ---
m_2 = np.array([55, 3, 5])

```

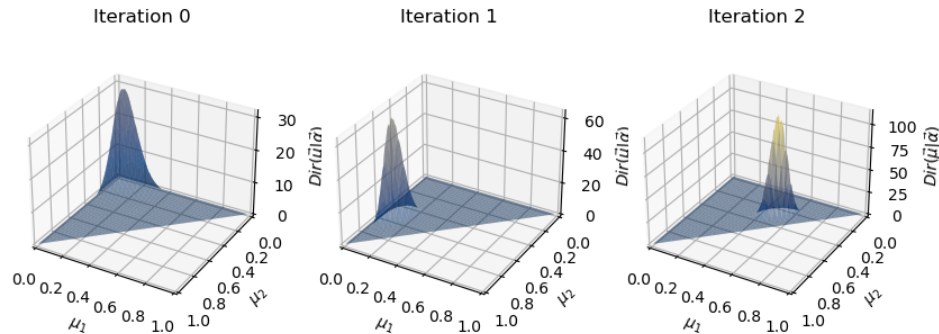


```

Z_history_matrix, alpha_2, _ = compute_bayesian_update(X, Y, alpha_1, m_2, 2,
↳Z_history_matrix)

# Plot using the given starter function
plot_K3_Dirichlet_dist(X, Y, Z_history_matrix)## Please write and run your code_
↳here
# Please add cells as needed

```



1.4 Reflection Questions

Once you are confident you implemented the Bayesian update correctly, check your work in Part 2 of this assignment by answering the following questions. Once you have completed that assignment, please look at the solution code to see how yours compares.

1.4.1 Questions 1, 2, and 3, respectively:

After all iterations of the algorithm were complete, what were the estimated values of μ_1 , μ_2 , and μ_3 that corresponded to the maximum value of the posterior? Please express your answers to three decimal places.

Hint 1: you might want to use a function like the NumPy function `argmax()`. This function won't quite work on it's own. Take a look at [the examples in the `argmax\(\)` documentation](#) to see what other functions might be useful to use in tandem with this function.

Hint 2: About half of the values in our dirichlet distribution matrix are NaN. The `argmax()` function will return NaN over other values. Look at similar functions in the table of contents on the left of the documentation of the `argmax()` function.

Hint 3: Think carefully about which vertex (x, y, z?) corresponds to μ_1 .

```

[8]: ## Please write and run your code here
# Please add cells as needed
# Exact analytical mode calculation for Dirichlet distribution

```

```

alpha_final = np.array([62, 36, 22])
K = len(alpha_final)
denominator = np.sum(alpha_final) - K

exact_mu1 = (alpha_final[0] - 1) / denominator
exact_mu2 = (alpha_final[1] - 1) / denominator
exact_mu3 = (alpha_final[2] - 1) / denominator

print(f"Exact Analytical mu_1: {exact_mu1:.3f}")
print(f"Exact Analytical mu_2: {exact_mu2:.3f}")
print(f"Exact Analytical mu_3: {exact_mu3:.3f}")

```

```

Exact Analytical mu_1: 0.521
Exact Analytical mu_2: 0.299
Exact Analytical mu_3: 0.179

```

```
[ ]: 0.521
```

```
[ ]: 0.299
```

```
[ ]: 0.180
```

1.4.2 Questions 4, 5, and 6, respectively:

In the last video you watched, Peter explained that the mean of the Dirichlet distribution was a function of the hyperparameters, such that $\mathbb{E}[\mu_k] = \frac{\alpha_k}{\alpha_0}$. What are our analytic solutions for μ_1 , μ_2 , and μ_3 , based on the values of $\vec{\alpha}$ after all iterations were complete?

```

[10]: ## Please write and run your code here
      # Please add cells as needed
      # Calculate total sum of hyperparameters after all iterations
      alpha_0_sum = np.sum(alpha_2)

      # Calculate expected values (analytic means)
      E_mu1 = alpha_2[0] / alpha_0_sum
      E_mu2 = alpha_2[1] / alpha_0_sum
      E_mu3 = alpha_2[2] / alpha_0_sum

      # Display results rounded to three decimal places
      print(f"Question 4 (Analytic Mean mu_1): {E_mu1:.3f}")
      print(f"Question 5 (Analytic Mean mu_2): {E_mu2:.3f}")
      print(f"Question 6 (Analytic Mean mu_3): {E_mu3:.3f}")

```

```

Question 4 (Analytic Mean mu_1): 0.517
Question 5 (Analytic Mean mu_2): 0.300
Question 6 (Analytic Mean mu_3): 0.183

```

[]: 0.517

[]: 0.300

[]: 0.183

[]:

1.4.3 Question 7:

How well did our estimate align with the expected value calculation? What could account for the difference in the two cases?

[]: Our grid estimates align closely **with** the expected value calculation. The minor ↪ numerical differences occur because the Dirichlet distribution **is** inherently ↪ asymmetric (skewed) due to unequal alpha parameters. For skewed probability ↪ distributions, the mode (maximum value point) **and** the mean (expected value) ↪ are **not** identical. As more data **is** gathered **and** the **sum** of alpha increases, ↪ the distribution becomes more symmetric, causing these two metrics to ↪ converge.

1.4.4 Question 8:

How did the variance of the Dirichlet distribution change as the iterations progressed? In your own words, what does that tell us?

[]: The variance of the Dirichlet distribution decreased significantly **as** the ↪ iterations progressed. This reduction tells us that our statistical ↪ uncertainty **is** shrinking **as** new data **is** incorporated. Visually, this ↪ manifests **as** a broad, flat probability surface **in** the initial prior ↪ sharpening into a highly localized, narrow spire by the final ↪ update ↪ indicating high confidence **in** our parameter estimates.